

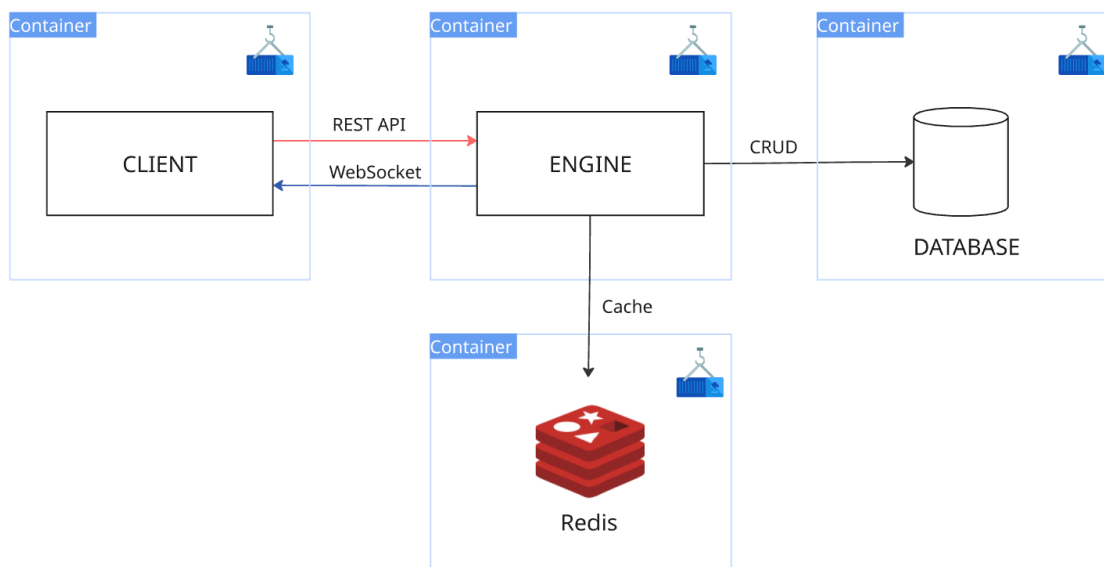
Distribuirani računarski sistemi

2025/2026

Platforma za učenje

Potrebno je implementirati platformu koja simulira osnovne funkcionalnosti obrazovne platforme za učenje. Sistem se sastoji od četiri komponente (slika 1):

1. **Korisnički interfejs** (*Client*)
2. **Servis za obradu zahteva i podataka** (*Engine*)
3. **Baza podataka** (*Database*)
4. **Keš baza podataka** (*Redis*)



Slika 1 Arhitektura sistema

1. Client

Korisnički interfejs je React ili Angular veb aplikacija koja ostvaruje komunikaciju sa **Engine** servisom putem **REST API** poziva, a za potrebe real-time funkcionalnosti koristi **WebSocket**.

2. Engine

Python Flask aplikacija služi za opsluživanje klijenta i direktnu komunikaciju sa bazom podataka. Zadužena je za obradu svih REST API zahteva koje klijent šalje, kao i za emitovanje real-time događaja.

3. Database

Baza podataka je centralna komponenta zadužena za čuvanje svih podataka u sistemu. U okviru projekta potrebno je odabrati odgovarajući tip baze — SQL ili NoSQL — u skladu sa prirodom i strukturom podataka.

4. Keš baza podataka

Redis (*Remote Dictionary Server*) je in-memory key-value baza podataka koja se koristi kao keš baza podataka. Čuva često korišćene podatke u RAM-u radi brzog pristupa.

Specifikacija zadatka

Autentifikacija

Korisnik se na platformu prijavljuje unošenjem kredencijala:

- **Email**
- **Lozinka**

Neophodno je implementirati autentifikaciju zasnovanu na sesijama (*session based authentication*). Nakon uspešne prijave za korisnika se kreira sesija koja se čuva u **keš bazi podataka**. Ograničiti trajanje sesije na jedan sat.

Omogućiti korisniku odjavu sa sistema.

Upravljanje korisnicima

Novi korisnički nalog može da kreira korisnik sa ulogom *administratora*. Prilikom kreiranja novog korisničkog naloga, unosi se:

- **Ime**
- **Prezime**
- **Email**
- **Uloga** (*profesor* ili *student*),
- **Datum rođenja**
- **Pol**
- **Država**

- **Ulica**
- **Broj**

Administrator može da izlista sve korisnike platforme i da briše korisničke naloge.

Korisnik, bilo da je *profesor* ili *student*, može da izmeni sve podatke o svom korisničkom profilu, kao i da doda sliku za svoj korisnički profil.

Upravljanje kursevima

Zahtev za kreiranje novog kursa šalje korisnik sa ulogom *profesor*. Zahtev se sastoji od:

- **Naziva kursa**
- **Opisa**

Zahtev stiže *administratoru* koji može da odbije zahtev ili da ga prihvati. *Profesor* koji je poslao zahtev se obaveštava putem imejla da li je njegov zahtev prihvaćen ili odbijen. *Profesoru* treba da se prikaže trenutno stanje zahteva za kreiranje kursa: **u obradi**, **prihvaćen** ili **odbijen**. *Administratoru* omogućiti da zahteve za novim kursom vidi u realnom vremenu upotrebom **Web Socket-a**.

Kada je kurs kreiran, *profesor* može da izmeni informacije o kursu, obriše kurs, doda *studente* na kurs, okači materijal za učenje (*pdf* fajl) i kreira nove zadatke.

Upravljanje zadacima

Zadatke za određeni kurs može da doda samo *profesor* koji je kurs i kreirao. Prilikom kreiranja novog zadatka profesor unosi:

- **Naziv zadatka**
- **Opis problema**
- **Krajnji rok za predaju**

Studentima koji su upisani na taj kurs stiže imejl poruka da je dodat novi zadatak u okviru tog kursa kao i krajni rok za izradu.

Studenti mogu da pregledaju materijal unutar kursa i da predaju svoje rešenje zadatka. Rešenje se predaje u .py formatu.

Npr. unutar kursa *Data Structures and Algorithms in Python* postavljen je zadatak *Graph Algorithms - Dijkstra's*. Student svoje rešenje predaje kao *dijkstra_implementation.py*.

Upravljanje rešenjima

Profesor u okviru kursa može da vidi sva predata rešenja za zadatke koje je kreirao. Omogućiti *profesoru* da oceni rešenje i ostavi komentar.

Napomene

- Neophodno je imati validaciju kako na klijentskoj strani, tako i na strani servera.
- Lozinke se **moraju** čuvati hešovane, nikako u čitljivom formatu.
- Prilikom odabira tipa baze podataka prvo istražite prednosti i mane SQL i NoSQL (nikako ne koristiti SQLite).
- Istražiti načine organizacije projekta kako u React/Angular, tako i u Flask-u. (struktura foldere)
- Ukoliko se odlučite za React, koristiti Vite.js alat.
- Koristiti ORM za komunikaciju sa bazom podataka (npr. SQLAlchemy za SQL baze).
- Koristiti ODM za komunikaciju sa bazom podataka (npr. PyMongo za MongoDB).
- U okviru projekta neophodno je na smislenom mestu upotrebiti procese.
- Koristiti virtuelno okruženje (venv, pipenv...).

Na odbranu projekta doći sa pripremljenim podacima za testiranje.

Način ocenjivanja

Projekat se ocenjuje prema sledećem

1. Aplikacija je funkcionalna i postoji Flask aplikacija – **51 poen**
 - a. Flask aplikacija + Baza podataka
2. Zaseban UI projekat (React, Angular...) koji komunicira sa Engine-om – **10 poena**
3. Implementacija keš baze podataka – **14 poena**
4. Koriscenje procesa prilikom implementacije – **10 poena**
5. Dockerizacija aplikacije (moraju **sve** komponente biti dockerizovane) - **10 poena**
6. Pokretanje na više računara – **5 poena**

BROJ OSVOJENIH POENA SE MNOŽI SA 0.9